

* Maven:- It is a powerful, open-source build automation and project management tool primarily used for Java-based software development.

It allows you to focus on business logic rather than infrastructure and build configurations.

Why we need Maven?

- Challenge of Sharing Code: Manually sharing projects by zipping folders or emailing a class files is inefficient and error-prone. It often leads to missing dependencies, incorrect folder structures, & broken references.

Solution: Maven revolves around concept of JAR files, which acts as packages to store compiled code, resources and metadata in a single, shareable format.

JAR (Java Archives)

- Dependencies Management: Real-~~time~~ projects need 100+ external libraries. And also, these libraries depend on other libraries, like transitive dependencies ($A \rightarrow B \rightarrow C$), so it makes very difficult to manage them.

Maven resolves this automatically by reading your [pom.xml] and downloading full dependency tree from Maven Central.

Date.....

- Version Conflicts: Without a tool, it's easy to accidentally mix incompatible versions. Maven pins exact versions in POM and resolves conflicts, eliminating "works on my machine" issues.
- No Standard Project Structure: Every developer used a different folder layout, making it hard for new team members to navigate a project. So, Maven enforces [convention-over-configuration] structure which everyone has to follow.

Basic Structure of Maven Project:-

src/main/java	Your main java source files (package hierarchy like groupid)
src/main/resources	Non-code files (e.g. XML configs, application properties)
src/test/java	Unit test source code (mirrors main package structure)
src/test/resources	Test-only config/data files
pom.xml	Project Object Model - defines dependencies, plugins and build config
target/	Auto-generated by Maven - contains compiled classes, final JAR, and intermediate build files.

*POM XML :- (Project Object Model) is the core configuration file of any Maven project. It's an XML file that tells Maven what the project is, what it needs, and how to build it.

It has many sections in it:-

1. <project> - Root Element. The top-level wrapper. Everything comes inside it.

2. <modelVersion> - Always 4.0.0. It identifies the POM schema version.

3. <parent> - Declares parent POM from which this project inherits configuration.

<parent>

<groupId> [Parent's group] </groupId>

<artifactId> [Parent's artifact] </artifactId>

<version> [Parent's version] </version>

<relativePath> [Filesystem path] </relativePath>

</parent>

4. Project Coordinates (GAV)

<groupId>

<artifactId>

<version>

<packaging> [Output types: jar, war, ear, pom]

5. Project Metadata - It is optional. Here we can add many tags for different functions. But, we cannot create custom tags.

6. <properties> - These are reusable variables. They can be used anywhere in POM by using `${key}`. We create key-value pairs in this. Also, we can create custom tags/properties here.

7. <dependencies> - It contains external libraries. Each <dependency> declares one library. Inside this, we specify metadata/tags of library. Like, <groupId>, <artifactId>, <version>, <scope>, <optional>, etc.

8. <dependencyManagement> - This does not add dependencies. It only declares versions so child modules or <dependencies> entries can omit <version>.

9. <repositories> - It tells from where to download dependencies.

10. <pluginRepositories> - This tells from where to download Plugins.

11. <distributionManagement> - This tells where to upload our artifacts. mainly used during [mvn deploy] to publish our artifact to somewhere.

12. <build> - It is very important. It tells how our project will be built.

Date.....

(a.) `<finalName>` - This overrides the default artifact name `[${artifactId}-${version}]`.

(b.) `<resources>`/`<testResources>` - It controls which files are copied to classpath and whether they're filtered.

(c.) `<pluginManagement>` - It declares plugin versions and configurations that child modules will inherit when they reference the plugin. This does not activate the plugin.

(d.) `<plugins>` - Active Build Plugins

(e.) `<extensions>` - Adds build extensions (e.g. for custom packaging types).

13. `<reporting>` - It configures plugins for [maven site]. This is used during project documentation.

14. `<profiles>` - In this, we can define different build configurations for different environments.

15. `<modules>` - This is for parent POMs that aggregate sub-projects.

Date.....

When maven download repositories, then it have 3 options:

Local Repository → maven downloads all dependy/repository from central server and store it in [m2] folder for faster access so that maven doesn't have to download them again.

Central Repository → This is the public, primary remote server where maven looks for libraries if they aren't available in your local cache folder.

Remote/Company Repositories → Organizations often set up private, custom repositories to host internal libraries or verified versions of external dependencies, which can be configured by overriding the repository settings in pom.xml.

* Workflow of Dependencies

1. Maven reads your pom.xml.
2. Checks your Local Repository (m2 folder) first.
3. If not found, it connects to the Central (or specified remote) repository to download them.
4. Stores the file in the Local Repository and adds it to your project's class path.

* Maven Build Lifecycle :-

Maven standardizes the build process by defining a clear lifecycle, which consists of a series of phases that executes in a specific order.

1. Clean :- Cleans the project by removing files generated during previous build (deleting /target folder).

2. Default :- The most critical lifecycle, covering the entire process from compilation to packaging & deployment.

Key Phases :-

- Validate = Checks that the project is correct and all necessary information is available.
- Compile = Compiles the source code into byte code (.class files) in target/classes.
- Test = Runs unit tests from src/test/java using a compatible testing framework.
- Package = Bundles the compiled code into a distributable format, like a JAR or WAR file.
- Verify = Runs integration ^{tests} ~~checks~~ and code quality checks.
- Install = Copies the packaged file and its POM into your local Maven repo (.m2 folder), making it available for other projects on your machine.
- Deploy = Uploads the final package to a remote repo for sharing across

Date.....

teams or CI/CD pipeline can pull it.

3. Site :- Generates the project's HTML documentation website into target/site/.

~~***~~ When you execute a later phase (like install), Maven automatically executes all preceding phases (validate, compile, test, package, verify) in order, ensuring the project is built correctly every time.